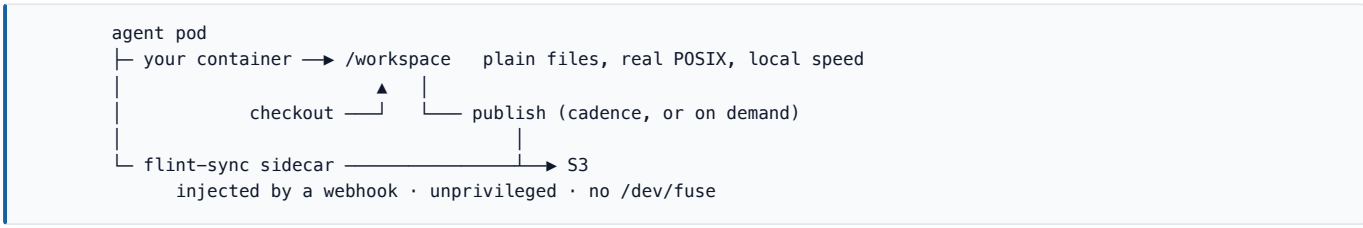
flint-lean on a Kubernetes cluster

Give every agent pod its own workspace: **plain local files** checked out of your S3 bucket at pod start, published back on a cadence — or when the agent says so. No FUSE, no NFS, no privileged pods, no mount to wedge.



The trade, stated up front: the workspace is a **local copy**. It must fit the pod's disk, one writer owns it at a time, and other readers see the last published boundary rather than your last write. If that is not your shape, see when not to use lean.

Everything below was run end to end on 2026-08-26 against the published chart. Commands are copy-pasteable, not illustrative.

Before you start

An S3 bucket	must already exist. **Versioning on** if you want gated mode — it is refused without it. Nothing here creates or deletes buckets.
Kubernetes	**1.29+** — the sidecar is injected as a *native sidecar* (`initContainer` with `restartPolicy: Always`), which is 1.29 or newer.
Tools	`kubectl` , `helm` 3.8+ (OCI support)
Credentials	see credentials — the agent container never holds one

Nothing is installed on your nodes. The sidecar is an ordinary unprivileged container.

---

1. Install the operator

```
kubectl create namespace flint-system

helm install flint-lean \
  oci://registry-1.docker.io/dilipdalton/flint-lean --version 0.3.0 \
  -n flint-system
```

That installs three things: the `FlintLeanWorkspace` CRD, the operator, and a mutating webhook. The webhook provisions **its own TLS certificate and `MutatingWebhookConfiguration` at startup** — there is no cert-manager dependency and nothing to pre-create.

It pulls two images and nothing else:

IMAGE	PULLED BY	WHAT RUNS
`dilipdalton/flint-lean-operator`	the `flint-system` deployment	`flint-lean-operator` — the controller and webhook
`dilipdalton/flint-sync`	every opted-in agent pod	the injected sidecar

flint-lean does not install or require flint-lite. No CSI driver, no NFS hub, no `FlintShare` CRD — the chart bundles exactly one CRD and its RBAC covers only `flintleanworkspaces` . (The operator image is the same build artifact as `flint-lite-operator` , republished under the lean name from an identical digest: one image, two names, so nothing is duplicated and nothing called "lite" appears in a lean install.)

Check it came up:

```
kubectl -n flint-system get pods
# flint-lean-59d5b4f886-s5hp8   1/1   Running

kubectl -n flint-system logs deploy/flint-lean | tail -3
# webhook cert generated and stored in flint-lean-webhook-cert
# mutating webhook flint-lean-inject applied
# flint-lean-operator: watching FlintLeanWorkspace
```

> **If `helm install` fails with `permission denied` on `~/Library/Caches/helm`**, something ran `helm` as root with your `HOME` and took ownership of your cache. Either `sudo chown -R "$USER" ~/Library/Caches/helm`, or run with a private cache: `HELM_CACHE_HOME=/tmp/helmcache helm install …`

Credentials

The operator does bucket-admin work (posture probes, a stale-upload sweep) under its own identity. Each workspace's sidecar uses the credential its CR names. **The agent container is given neither** — the webhook stamps the secret onto the sidecar container only, and process namespaces are not shared, so a compromised agent cannot read it.

```
kubectl -n flint-system create secret generic s3 \
  --from-literal=AWS_ACCESS_KEY_ID=... \
  --from-literal=AWS_SECRET_ACCESS_KEY=... \
  --from-literal=AWS_REGION=us-west-1

# and the same secret in the namespace your agents run in
kubectl -n agents create secret generic s3 --from-literal=...
```

Point the operator at it by reinstalling (or `helm upgrade`) with `--set operatorCredentialsSecret=s3`, or leave it empty to use the pod's ambient chain (IRSA and friends).

The keys must be named `AWS_*` **verbatim** — they are passed to the sidecar as environment.

> **Plan of record:** a project-scoped S3 proxy holds the real credentials and no pod stores one at all. Until that lands, `credentialsSecretRef` is the interim posture.

---

2. Create a workspace

One CR per project subtree.

```
apiVersion: flint.io/v1alpha1
kind: FlintLeanWorkspace
metadata:
  name: proj1
  namespace: agents
spec:
  projectId: team-a/proj1        # durable claim identity
  bucket: my-bucket
  keyPrefix: tenants/proj1       # the subtree this workspace owns
  credentialsSecretRef: s3
  floorSecs: 60                  # publish cadence — and the RPO
```

```
kubectl apply -f workspace.yaml
kubectl -n agents get flintleanworkspace proj1 -o yaml | grep -A6 conditions
```

You want `BoundaryModeAccepted: True`. `BoundaryModeActive: Unknown` with reason `NoLiveSidecar` is normal for a workspace at rest — nothing holds the lease until a pod starts.

---

3. Opt a pod in

One label. That is the whole integration.

```
apiVersion: v1
kind: Pod
metadata:
  name: agent-1
  namespace: agents
  labels:
    flint.io/lean-workspace: proj1     # ← the webhook keys on this
spec:
  containers:
    - name: agent
      image: your-agent:latest
      workingDir: /workspace
```

The webhook injects the sidecar, adds the workspace volume, and mounts it into **every** container in the pod. The sidecar runs the checkout *before* your container starts — the injected `startupProbe` gates it — so your first line of code sees a complete tree.

```
kubectl -n agents get pod agent-1
# agent-1   2/2   Running        ← 2/2 = your container + the sidecar
```

If the workspace is missing or refused, the pod is **rejected at admission** rather than started against an empty directory.

> **Do not declare your own volumeMount at `/workspace`** — the injection would collide and admission fails. If your image needs that path, move flint's with `spec.mountPath: /flint` on the CR.

---

4. Publish on demand (optional, recommended)

Cadence alone forces a bad choice: publish often and waste work, or publish rarely and let a reader see a half-finished tree. Instead, let the agent declare a coherent point when it finishes a unit of work:

```
# in the agent container
printf '{"nonce":"task-42"}' > /workspace/.flint/publish

# wait for the answer
until [ -f /workspace/.flint/publish.ack ]; do sleep 1; done
cat /workspace/.flint/publish.ack
# {
#   "status": "ok",
#   "nonces": [ "task-42" ],
#   "seq": 1,
#   "manifest_etag": "\"caa5498c...\"",
#   "boundary": "sentinel",
#   "completed_unix": 1787772547,
#   "report": { "uploaded": 0, "deleted": 0, "parked": 0, ... }
# }
```

`uploaded: 0` here just means nothing had changed since the last boundary — the point the agent declared is already true, which is still an honest `ok`.

`status: "ok"` means **the named boundary is in the bucket** — not queued, not scheduled. `refused-fenced` means this sidecar lost the lease and is telling you rather than leaving you waiting.

Costs nothing when unused: measured at 20 bucket requests per 22 s idle with the verbs off, and 20 with them on.

---

5. Verify

```
aws s3 ls s3://my-bucket/tenants/proj1/ --recursive
# tenants/proj1/.flint/lean/claim
# tenants/proj1/.flint/lean/epoch
# tenants/proj1/.flint/lean/manifest
# tenants/proj1/files/...          ← your tree lives here
```

The `files/` prefix is the workspace. `.flint/lean/` is control state. To see which clock installed the current boundary:

```
aws s3api head-object --bucket my-bucket \
  --key tenants/proj1/.flint/lean/manifest \
  --query 'Metadata."flint-boundary-source"'
# "cadence" | "sentinel" | "quiescence" | "drain" | ...
```

What it costs

Measured floors (loopback; a latency-bound proxy moves these):

bytes	checkout **3.3 s/GiB** · publish **8.0 s/GiB**
100k files	checkout **49.5 s** · first publish **65 s** · idle tick **1.85 s**
1M files	checkout **7 m 05 s**; manifest **264 MiB** at ~277 B/entry
idle	~5 bucket requests per tick, per workspace

The file count binds before the byte count — the manifest is exactly linear in entries, which is why the v1 cap is ~250k files.

When not to use lean

YOU NEED	USE
a tree too large for the pod's disk, or lazy access to a slice of a huge dataset	FUSE (`docs/flint-fuse-architecture.html`)
two pods writing one tree and seeing each other live; byte-range locks; shared sqlite/git across pods	the hub (flint-lite)
readers that cannot tolerate seeing the last boundary instead of the last write	the hub

All three front ends share one bucket format and are mutually convertible, so choosing lean now does not strand the data.

For agents collaborating on code, do **not** reach for a git-over-S3 remote: use plain `git` against a real git host and let flint-lean keep the workspace durable. See `docs/flint-lean-git-workflow.md`.

Tearing down

```
kubectl -n agents delete pod agent-1
kubectl -n agents delete flintleanworkspace proj1   # leaves bucket data intact
helm uninstall flint-lean -n flint-system
```

Deleting the CR does not delete your data. The bucket subtree is the durable artifact; a new workspace pointed at the same `keyPrefix` picks it up.